

Take-Home Assignment: QA Automation Agent

Duration: 5 days

Overview

Build a QA automation agent that autonomously pulls tickets from Jira/docs/sheets, generates or fetches associated test cases, logs them in a structured format, and then executes those test cases against a target application — handling both frontend (browser-based) and backend (API-based) test cases.

What You'll Build

1. The QA Agent (the main deliverable)

An automation agent that does the following, end-to-end:

- **Connect to Jira/Docs/Sheets:** Authenticate against the Jira/Docs/Sheets REST API and fetch tickets (issues).
- **Log tickets:** Persist the fetched tickets somewhere durable — a database, a file, or an Google Doc/Sheet. Your choice; justify it briefly in the README.
- **Handle test cases:** If a ticket already has linked test cases (in docs/sheets/etc, or even just described in the ticket description), pull them. If it does not, the agent should generate test cases from the ticket description using an LLM.
- **Classify each test case:** Determine whether it is a frontend test (UI interaction) or a backend test (API call). The agent should do this autonomously.
- **Execute each test case:** Frontend tests run against the deployed app URL via a browser agent. Backend tests run as HTTP requests against the API with assertions on response status, body shape, and (optionally) database state.
- **Report results:** Capture pass/fail, logs, screenshots (for frontend), and request/response payloads (for backend). Write results back to your google doc/sheet as a comment on the ticket, and into your log (sheet/DB).

2. The target application (provided)

We provide a deployed app for your agent to test against:

- Deployed URL: <https://qa-assignment-steel.vercel.app/>
- Domain: a small todo app (auth, per-user todos with priority and due date, list/filter/edit/delete). All users have the same hierarchy.

Seeded accounts created for your convenience(use these, or register new ones via the UI):

email	password
admin1@example.com	password123
bob@example.com	password123
admin2@admin.com	adminadmin

No API documentation is provided. Discover the app's features and endpoints the way a QA hire onboarding to an unfamiliar product would: use the app in a browser, watch the DevTools network panel, infer request/response shapes from what you see. Your tickets (Section 3) should describe what you've learned, in your own words.

Tooling Pointers

Implementation is entirely up to you.

For frontend execution, two open-source options worth looking at are Stagehand (browserbase/stagehand) and Browser Use (browser-use/browser-use). Both are LLM-driven browser agents. Pick one and justify your choice in the README.

For backend execution, the approach is yours to design.

Deliverables

Submit the following by EOD on 31 May 2026:

1. GitHub repository containing the agent code. Add tanay@tri-pod.com and yaroslav.kutukov@gmail.com as a collaborator.
2. README covers how to run it, the architecture, key decisions, and known limitations.
3. A short video walkthrough (5–10 minutes) showing the agent running end-to-end. Upload to YouTube as an unlisted video and share the link here and in the README.md.
4. Access to the Jira/google sheet/docs so we can verify outputs. Give access to tanay@tri-pod.com and yaroslav@tri-pod.com
5. **Daily progress log:** at the end of each day, append a short update to this same document (in the Daily Progress Log section at the bottom). Cover your thought process, what you built, what's blocking you, and what you're tackling next.

Evaluation Criteria

Criterion	What we're looking for
System design	Clean separation of concerns. Sensible flow between Test Cases → normalize → execute → report. Reasonable choices around state and queuing.
Agent integration	How well you've wired up the LLM-driven browser agent. Handling of flakiness, retries, screenshots, and failure modes.
Backend testing approach	Quality of structured test generation, robustness of assertions, handling of auth and stateful endpoints.
Pragmatism	Did you ship a working end-to-end demo, or were you only able to complete selected parts of the assignment? We strongly prefer the former.
Communication	README clarity, video walkthrough, and how well you explain your trade-offs and what you'd do differently with more time.

Logistics

- **Start date: June 3, 12:00 PM**
- **Deadline: June 8, 12:00 PM. Your access to this google doc would be cut off after that. Any commits after then would not be regarded.**
- **Tooling budget:** You'll receive \$60 worth of Anthropic API credits, which is the only AI tooling provided. Use it for whatever you need — test case generation, classification, the browser agent. Stay within budget. Use models appropriately
- Anthropic API Key:
sk-ant-api03-EMODvBA3TZ_dKdBtXDSLikOYUrgqVfDTAZUfRIHHgEE-qfNYFdnAvBpDj6OmHIDIfD_9AiO3BWzh8rLHj8MusA-6GYpkwAA

Daily Progress Log

Append a short update at the end of each day. What your approach would be, what you built, what's blocking, what's next.

Format:

Day X [Date] —

Description

...

